

Подключение нового ресторана к SeatMap

Версия инструкции: 6 сентября 2026 года

1. Что создаётся для нового ресторана

Для каждого нового клиента используются:

- тот же GitHub-репозиторий и общий backend на Render;
- новый tenant в `backend/CasaDiFratelli.Api/appsettings.json`;
- отдельная PostgreSQL-база данных;
- отдельный Vercel Project из папки `frontend`;
- отдельный домен ресторана;
- отдельные владелец, отправители Resend и настройки бренда.

Копировать frontend в новую папку или создавать отдельную ветку не нужно. Один frontend получает название, логотип, контакты и правила резервирования через `GET /api/branding`. Отдельный frontend-проект нужен для собственного домена и публичного контента ресторана.

2. Собрать данные до начала работ

Попросите у ресторана название, домен и доступ к DNS, телефон, email, адрес на BG/EN/RU, рабочее время, часовой пояс, логотип, фотографии, социальные ссылки, схему столов, меню, список ролей, тариф Basic/Pro и email владельца.

Пример технических имён для Restaurant Bella:

```
tenant id: restaurant-bella
slug: restaurant-bella
connection key: RestaurantBella
owner email key: BELLA_ADMIN_EMAIL
owner password key: BELLA_ADMIN_PASSWORD
domain: restaurantbella.bg
```

Tenant ID: нижний регистр, латинские буквы, цифры и одиночные дефисы. После запуска его лучше не менять: он участвует в email idempotency, настройках и пути резервных копий.

3. Создать отдельную PostgreSQL-базу

В Render откройте Dashboard → New → PostgreSQL:

1. Назовите базу, например `seatmap-restaurant-bella`.
2. Выберите тот же регион, где работает backend.

3. Включите production plan и резервные копии Render.
4. Скопируйте Internal Database URL.

Нельзя использовать базу Casa или другого ресторана. Backend остановит запуск, если две записи указывают на одинаковые host, port и database.

В Environment общего backend-сервиса Render добавьте секрет:

```
ConnectionStrings__RestaurantBella=<Internal Database URL новой базы>
```

Не записывайте connection string в GitHub, `appsettings.json` или сообщения.

4. Добавить tenant в backend

В `backend/CasaDiFratelli.Api/appsettings.json` добавьте объект в массив `Tenancy:Tenants`, не удаляя Casa:

```
{
  "Id": "restaurant-bella",
  "Name": "Restaurant Bella",
  "Slug": "restaurant-bella",
  "Domain": "restaurantbella.bg",
  "FrontendUrl": "https://restaurantbella.bg",
  "FrontendOrigins": [
    "https://restaurantbella.bg",
    "https://www.restaurantbella.bg",
    "https://restaurant-bella.vercel.app"
  ],
  "SeedDefaultMenu": false,
  "AdminEmailConfigurationKey": "BELLA_ADMIN_EMAIL",
  "AdminPasswordConfigurationKey": "BELLA_ADMIN_PASSWORD",
  "DatabaseMode": "DedicatedDatabase",
  "ConnectionStringKey": "RestaurantBella",
  "IsActive": true
}
```

Origin указывается как схема + host, без пути и завершающего `/`. Домены, ID, slug, connection key и ключи администратора должны быть уникальными.

В Render добавьте секреты первого владельца:

```
BELLA_ADMIN_EMAIL=owner@restaurantbella.bg
BELLA_ADMIN_PASSWORD=<уникальный длинный пароль>
```

Владелец создаётся, только если в новой базе ещё нет администраторов. После первого входа пароль следует сменить.

5. Настроить email ресторана

В Resend добавьте и подтвердите домен ресторана. Разделите транзакционные письма и маркетинг: `reservations@mail.restaurantbella.bg` и `offers@mail.restaurantbella.bg`.

В Environment backend-сервиса Render добавьте:

```
Tenancy__Email__restaurant-bella__RESEND_API_KEY=<ключ Resend>
Tenancy__Email__restaurant-bella__FROM_EMAIL=Restaurant Bella
<reservations@mail.restaurantbella.bg>
Tenancy__Email__restaurant-bella__MARKETING_FROM_EMAIL=Restaurant Bella
<offers@mail.restaurantbella.bg>
Tenancy__Email__restaurant-bella__REPLY_TO_EMAIL=hello@restaurantbella.bg
Tenancy__Email__restaurant-bella__UNSUBSCRIBE_EMAIL=hello@restaurantbella.bg
Tenancy__Email__restaurant-bella__ADMIN_EMAIL=owner@restaurantbella.bg
Tenancy__Email__restaurant-bella__VAPID_SUBJECT=mailto:owner@restaurantbella.bg
```

`BELLA_ADMIN_EMAIL` создаёт владельца. Tenant-переменная `ADMIN_EMAIL` получает уведомления. VAPID public/private keys сейчас общие, subject — отдельный. Глобальные Casa email-переменные не копируйте: другой tenant специально не использует Casa sender как fallback.

6. Проверить и развернуть backend

Локально из корня проекта выполните:

```
dotnet build backend/CasaDiFratelli.Api/CasaDiFratelli.Api.csproj
dotnet run --project
backend/CasaDiFratelli.TenancyChecks/CasaDiFratelli.TenancyChecks.csproj
```

Сделайте commit и push в `main`. Render проверит tenant-конфигурацию, подключится к базам, применит migrations и создаст владельца. Меню Casa не копируется, поскольку `SeedDefaultMenu=false`.

В Render logs должны появиться:

```
Tenant database initialized. TenantId=casa-di-fratelli
Tenant database initialized. TenantId=restaurant-bella
```

При ошибке не удаляйте миграции и не направляйте tenant на старую базу — сначала исправьте первую ошибку в логах.

7. Создать frontend на Vercel

Vercel → Add New → Project → импортировать тот же репозиторий `DISpace-Software/casa-di-fratelli`.

```
Project Name: restaurant-bella
Root Directory: frontend
Framework Preset: Vite
Build Command: npm run build
Output Directory: dist
Node.js Version: 22.x
```

Для Production, Preview и Development добавьте:

```
VITE_API_BASE_URL=https://<домен общего backend на Render>
```

Не добавляйте базу, Resend или admin password в Vercel: `VITE_*` публичны.

После первого deployment скопируйте точный `*.vercel.app`. Если он отличается от `FrontendOrigins`, обновите tenant и сначала передеплойте backend, иначе CORS правильно заблокирует frontend.

8. Подключить собственный домен

Vercel → Project → Settings → Domains:

1. Добавьте домен и при необходимости `www`.
2. Создайте DNS-записи, которые покажет Vercel.
3. Дождитесь Valid Configuration и HTTPS.
4. Выберите основной домен и redirect для второго варианта.

Оба адреса должны быть в `FrontendOrigins`. В ответе `/api/branding` заголовок `X-Tenant-Id` должен равняться `restaurant-bella`.

9. Заполнить ресторан через админку

Откройте `/admin`, войдите через первичные credentials и заполните:

- название, телефон, email, logo URL;
- адрес и рабочее время на BG/EN/RU;
- IANA timezone, например `Europe/Sofia`;
- Google Reviews, Facebook и Instagram;
- lead time, количество дней вперёд;
- последнее публичное/админское время;
- начало и окончание walk-in периода.

Затем настройте тариф, пользователей и роли, меню, залы/столы, события, маркетинг, а для Pro — склад и рецептуры.

10. Заменить контент ресторана

Runtime branding общий, но перед запуском проверьте индивидуальный контент:

- hero/gallery/chef/awards фотографии и тексты;
- ссылки доставки;
- privacy/legal тексты;
- `robots.txt` , `sitemap.xml` , web manifests;
- SEO meta и JSON-LD в `frontend/index.html` .

В будущем этот контент лучше вынести в CMS/tenant content API. Не создавайте долгоживущую копию всего frontend: она перестанет получать общие исправления.

11. Обязательный smoke test

1. Главная показывает правильные бренд, контакты и адрес.
2. Меню содержит только данные нового ресторана.
3. Тестовая резервация видна только в его админке.
4. Подтверждение приходит от его sender и с его брендом.
5. Владелец входит, меняет пароль и создаёт второго администратора.
6. Карта содержит правильные залы, столы и места.
7. Блокировка стола действует только выбранный рабочий день.
8. Birthday/marketing email используют его sender.
9. Push содержит его название.
10. Отчёты не содержат данные Casa.
11. Casa после deployment продолжает открывать свои меню, карту и админку.

12. Финальная безопасность

- Проверьте уникальность баз, ключей, admin secrets и senders.
- Включите PostgreSQL backups и persistent `BACKUP_DIRECTORY` .
- Не храните секреты в Git, Vercel frontend и документах.
- Включите MFA в GitHub, Render, Vercel, Resend и DNS.
- Ограничьте сотрудников ролями.
- Проверьте CORS со всех production-доменов.
- После регистрации прямых API-интеграций установите `Tenancy:RequireKnownTenant=true` для строгого routing.
- Настройте мониторинг Render и домена.

Короткий порядок

Данные клиента

- отдельная PostgreSQL
- tenant + secrets в Render
- Resend
- backend deploy/migrations
- новый Vercel Project из того же repo/frontend
- VITE_API_BASE_URL
- домен/DNS
- branding/menu/tables/users
- smoke test нового ресторана и Casa
- передача доступа и запуск